

ServiceNow Development with Script-Controlled ACL

Project Documentation

Restrict Record Access Based on Field Value

Team Leader: K. Kumutha

Team Members: R. Siva Dharshini, S. Vayshavi, M. Jasmin

Team ID: SWTID-2026-9038

Programme: Naan Mudhalvan – ServiceNow (Skill Wallet)

Date: 28 September 2026

1. Introduction

• **Project Title:** Script-Controlled ACL – Restrict Record Access Based on Field Value •

Team Members: K. Kumutha (Team Leader – PDI setup, table design and coordination);

R. Siva Dharshini (roles and users); S. Vayshavi (ACL script development); M. Jasmin (testing and documentation).

2. Project Overview

- **Purpose:** To restrict access to records of the custom table `u_institution_details` using an Access Control List (ACL) whose script checks the value of a field in the record. Users see only the records that are meant for their role, even though all of them open the same table. The goal is record-level security without creating separate tables or many extra roles.
- **Features:** Custom table with institution data; roles `bb1`, `bb2`, `bb3` and `bb4`; a record-type read ACL with an Advanced script; automatic filtering of lists, forms and reports; the same security for the REST API; testing with the Impersonate User feature; admin users keep full access.

3. Architecture

- **Frontend:** The ServiceNow user interface (Next Experience / Platform UI). Users open the list view and form view of `u_institution_details`. No custom client code is needed because the restriction is applied by the server before the data reaches the browser.
- **Backend:** The server-side ACL engine of ServiceNow. When a user requests a record, the engine evaluates the ACL conditions (table, operation, role) and then runs the Advanced script written in JavaScript. The script sets `answer = true` to allow or `answer = false` to deny.
- **Database:** The custom table `u_institution_details` stored in the ServiceNow instance database. Fields: Name (string), Location (string), Type (choice) and Access Level (string, holds the role name `bb1`–`bb4`). Each record is checked against the ACL on every read.

Flow: User request → ACL lookup for the table and operation → role check → script check of the Access Level field → allowed records returned, denied records hidden.

4. Setup Instructions

- **Prerequisites:** A free ServiceNow Personal Developer Instance (from developer.servicenow.com), an administrator login, a web browser (Chrome or Edge), and basic knowledge of JavaScript.
- **Installation:** No software installation is needed. Follow these steps in the instance:
 1. Log in to the PDI as administrator.
 2. Create the table `u_institution_details` (System Definition > Tables) and add the fields Name, Location, Type and Access Level.
 3. Create the roles `bb1`, `bb2`, `bb3` and `bb4` (User Administration > Roles).
 4. Create four test users and assign one role to each user.
 5. Add sample records with different Access Level values (`bb1` to `bb4`).

6. Elevate the role to security_admin, then open System Security > Access Control (ACL) and click New.
7. Set Type = record, Operation = read, Name = u_institution_details, enable Advanced and paste the script.
8. Save the ACL and test it with the Impersonate User option.

5. Folder Structure

- **Client:** There is no separate client folder. The user interface is provided by ServiceNow. The parts used are the list view (u_institution_details_list) and the form view (u_institution_details).
- **Server:** The server-side items are stored as records in the instance: the Table (System Definition > Tables), the Roles (sys_user_role), the Users (sys_user), and the ACL (sys_security_acl) which contains the script. All of them can be captured in one Update Set and exported as an XML file.

6. Running the Application

- **Frontend:** Open the instance URL, log in, type *u_institution_details.list* in the navigator filter and press Enter.
- **Backend:** No command is required because the ACL runs automatically on the server. To run a test, use the profile menu > Impersonate User, select a user (for example the bb1 user) and open the same list. Repeat for the other users.

The ACL script used:

```
answer = false;
// allow only if the record Access Level matches a role of the user if
(gs.hasRole(current.u_access_level))
answer = true;
```

7. API Documentation

ServiceNow exposes the table through the Table API, and the same ACL is applied to it, so a user gets only the permitted records through REST also.

- **GET /api/now/table/u_institution_details** – returns the list of records the user is allowed to read. Parameters: sysparm_limit, sysparm_query (for example sysparm_query=u_access_level=bb1), sysparm_fields. Example response: {"result":[{"sys_id":"...", "u_name":"Anna Institute of Tech", "u_access_level":"bb1"}]}
- **GET /api/now/table/u_institution_details/{sys_id}** – returns one record. If the ACL denies access, the response is 403 Forbidden or 404 Not Found.
- **POST /api/now/table/u_institution_details** – creates a record (needs a create ACL). **PATCH** and **DELETE** follow the write and delete ACLs. Request body example: {"u_name":"New College", "u_access_level":"bb2"}

8. Authentication

- **Authentication:** Users log in with a user name and password. The session is maintained by ServiceNow using session cookies. For REST calls, Basic Authentication or OAuth tokens are

Team SWTID-2026-9038 | Page 3

used.

- **Authorization:** Roles bb1, bb2, bb3 and bb4 are assigned to users. The ACL first checks that the user has a role required by the ACL, and then the script checks that the Access Level of the record matches a role held by the user. The Admin role can see all records.

9. User Interface

- List view shows only the records permitted for the logged-in user, and the row count changes with the user.
- Form view opens only for allowed records; a denied record shows a security message.
- The ACL form (System Security > Access Control) contains the Advanced checkbox and the Script field where the condition is written.

10. Testing

- **Strategy:** Manual functional testing using the Impersonate User feature. Each role is tested for list, form and REST access.
- **Test cases:** (1) bb1 user sees only bb1 records. (2) bb2 user sees only bb2 records. (3) bb3 and bb4 users see only their own records. (4) A user without any role sees nothing. (5) Admin sees all records. (6) Changing the Access Level of a record moves its visibility to the new role.
- **Tools used:** Impersonate User, the list filter, the REST API Explorer, and the System Logs to debug script errors.
- **Result:** All test cases passed and the list loaded without noticeable

11. Screenshots or Demo

servicenow					System Administrator
Institution Details [u_institution_details]					
All records					
Number	Name	Location	Type	Access Level	
IN0001	Ama Institute of Tech	Chennai	Engineering	bb1	
IN0002	Bharathi Arts College	Coimbatore	Arts	bb2	
IN0003	Cauvery Polytechnic	Trichy	Polytechnic	bb1	
IN0004	Delta Medical College	Madurai	Medical	bb3	
IN0005	Eastern Law School	Salem	Law	bb4	
IN0006	Global Business School	Chennai	Management	bb2	

1 to 6 of 6 rows

Figure 1 Table u_institution_details with records (admin view)

Team SWTID-2026-9038 | Page 4

servicenow		System Administrator
Access Control u_institution_details (read)		
Type	record	
Operation	read	
Name	institution Details [u_institution_details]	
Active	True	
Advanced	True	
Requires role	bb1, bb2, bb3, bb4	
Script	<pre> answer = false; // allow only if the record Access Level matches a role of the user if (gs.hasRole(current.u_access_level)) answer = true; </pre>	
Update		

Figure 2 ACL configuration with Advanced script

servicenow					impersonating: bb1 user
Institution Details [u_institution_details]					
all > Access Level = bb1					
Number	Name	Location	Type	Access Level	
INS0001	Anna Institute of Tech	Chennai	Engineering	bb1	
INS0003	Cauvery Polytechnic	Tiruchy	Polytechnic	bb1	

1 to 2 of 2 rows

Figure 3 List view for the bb1 user (only bb1 records)

servicenow					impersonating: bb2 user
Institution Details [u_institution_details]					
all > Access Level = bb2					
Number	Name	Location	Type	Access Level	
INS0002	Bharathi Arts College	Coimbatore	Arts	bb2	
INS0005	Global Business School	Chennai	Management	bb2	

1 to 2 of 2 rows

Figure 4 List view for the bb2 user (only bb2 records)

Demo link: Add your demo / GitHub link here.

12. Known Issues

- A mistake in the script can block valid users, so it must be tested after every change.
- A wrong or empty Access Level value in a record makes that record invisible to non-admin users.
- The security_admin role must be elevated to create or edit ACLs.
- Very complex scripts can slow down large tables.

13. Future Enhancements

- Extend the ACL to write, create and delete operations and to field-level

ACLs. • Move the logic into a Script Include so it can be reused on many tables.

- Add group-based, department-based and location-based conditions.
- Log denied access attempts and show them in a security dashboard.
- Apply the same approach to HR, finance and student data applications.